

LFX Coding Challenge: High-Precision Code Base Acceleration

Program: Broadening the RISC-V High Precision Code Base and Reach (MIT) **Candidate:** Valerii Platonov **Repository Architecture:** github.com/vpplatonov/risc-v-mentorship

1. Algorithmic Architecture Analysis

1.1. Tower of Hanoi (Structural Induction via Pure Recursion)

The implementation demonstrates a functional recursive engine. The mathematical problem of moving N disks is lower-bounded by $O(2^n)$ computational complexity.

- **Recursion Demonstration Point:** Inside `move_hanoi()`, the function processes the base case (`if n == 1`) to halt stack winding. For $N > 1$, it triggers a split execution flow, invoking itself recursively to handle sub-problems via the auxiliary storage boundaries. This logic maps to structural stack frames used in high-precision computation layers.

1.2. Conway's Game of Life (Deterministic Matrix Iteration)

The cellular automaton evaluates finite state transitions across a multi-dimensional matrix layout.

- **Iteration Demonstration Point:** The execution engine inside `run_conway_life()` relies on fixed double-nested loop configurations to evaluate pixel states. It guarantees deterministic array operations across rows and columns without data corruption or dynamic boundary leaks, mimicking array-vectorization steps described in *"Numerical Recipes: The Art of Scientific Computing"*.

2. Dynamic Verification Trace (Terminal Execution Logs)

2.1. Executing Recursive Component Target (Hanoi)

```
$ python3 src/python/run_challenges.py
--- TOWER OF HANOI (RECURSION ENGINE) ---
```

```

      |           |           |
      |           |           |
    [===]         |           |
    [=====]     |           |
=====
Peg A           Peg B           Peg C

```

2.2. Executing Iterative Component Target (Conway Grid)

```
$ python3 src/python/run_challenges.py conway
--- CONWAY'S GAME OF LIFE (ITERATIVE MATRIX) - GEN 5/10 ---
#
##
##
=====
```

3. Direct Traceable Verification Links

The raw implementations and structural test suites are accessible on the public version-control node:

- **Unified Automation Entrypoint Script:** [src/python/run_challenges.py](#)
- **Main Workspace Root Blueprints:** [src/python/](#)